NAME THE BIGGEST NUMBER

A Constructive Lean Contender Beyond Contender 5

Total interpretation, well-founded trees, and feedback through power towers

A reproducible Lean 4 contest entry
12 October 2026

Abstract

We adapt Cody Roux's constructive largest-number contest from Coq to Lean 4 and supply a concrete, executable-in-principle natural number, $N_\star$, together with a kernel-checked proof that it strictly exceeds the repository's Contender 5. The baseline is the original total interpretation of raw simply typed lambda-calculus syntax with natural-number recursion, maximized over all terms of depth at most 42. Its Lean translation preserves malformed terms, computational casts, reversed variable indices, and the original winner selection rule.

Our construction combines a cumulative envelope of this language with structural recursion on countably branching well-founded trees. An exact grafting identity turns substitution of trees into replacement of the base growth function. Higher function spaces and explicitly specified closure constructors supply the tree payload; score-dependent iteration supplies a quantitative guarantee independent of any ordinal interpretation. With $T(0) = 1$, $T(h+1) = 2^{T(h)}$, and $\Theta(s,0) = s$, $\Theta(s,k+1) = T(\Theta(s,k)+1)$, the checked result is

$$C_5 < N_\star, \qquad \Theta(C_5+1, C_5+1) \le N_\star.$$

All 32 audited declarations have empty transitive axiom lists. Fresh cumulative checking of all custom definitions and all comparison proofs fits the contest's 15-second and one-minute budgets on the recorded machine. The construction is an explicit submission, with no claim of optimality among all admissible Lean programs. Exact ordinal calibration of the Veblen-inspired syntax is not used; an explicit counterexample explains why such calibration cannot be assumed.

Pinned implementation: Lean 4.19.0. Upstream snapshot: `d56dce013e62`.
The archive contains the article, all Lean sources, an independent small-input model, reproduction scripts, source provenance, and validation records.

Contents

1 The proposed entry and the claim it establishes **2**

2 A precise Lean adaptation of the rules **2**

3 What Contender 5 actually denotes **3**
3.1 Raw syntax and depth . . . 3
3.2 A total semantic interpreter . . . 4
3.3 The selected maximum and the numerical envelope . . . 4

4 An independent inflationary base **5**

5 A constructive amplifier on well-founded trees **6**
5.1 Trees, evaluation, and substitution . . . 6
5.2 Explicit tree operators . . . 8
5.3 A dynamically chosen tower of function spaces . . . 10
5.4 Feedback and the final number . . . 11
5.5 A quantitative certificate independent of calibration . . . 11

6 The closed contender and its certified margin **13**

7 What the tree notation does and does not assert **14**

8 Validation and reproducibility **15**
8.1 What was checked . . . 15
8.2 Independent semantic checks . . . 16
8.3 Commands . . . 16

9 Constructivity, the diagonal barrier, and optimality **17**
9.1 Why the definition is a natural number . . . 17
9.2 Why one cannot simply maximize over all Lean definitions . . . 17
9.3 The meaning of “largest” . . . 17

10 Further research directions **18**

A Archive map and provenance **18**

B Complete computational amplifier **19**

C The base and final comparison proofs **21**
C.1 Base definitions . . . 21
C.2 Base proofs . . . 21
C.3 Closed definition . . . 22
C.4 Final certificates . . . 22

1 The proposed entry and the claim it establishes

The submitted number is the closed Lean definition

```
namespace BigNumber

def contender_6 : Nat := Amplifier.candidate base

end BigNumber
```

This short declaration names a fully specified construction; all dependencies are included and counted in the validation budget. The main proof is named `BigNumber.contender_5_lt_contender_6`. A stronger theorem, `BigNumber.contender_6_tower_bound`, establishes the boxed inequality in the abstract. Neither theorem evaluates C_5 or $N_\star$.

The lower bound has a direct reading. Start at $C_5 + 1$. Construct a tower of twos whose height is one greater than that starting value. Use the resulting integer, plus one, as the height of the next tower. Repeat this operation $C_5 + 1$ times. The resulting integer is a proved lower bound for the submitted number. The actual definition also contains substantial tree and higher-type structure that this estimate does not attempt to measure.

There are three distinct levels of assurance:

1. The mathematical definitions and proofs below explain the construction without requiring the reader to understand Lean tactics.
2. The supplied Lean files check the exact translated baseline, tree algebra, and numerical comparison with no axiom dependencies.
3. A source audit connects the translated baseline to the Coq source. This last connection is a reviewed translation, supported by independent small-input tests; it is not a formal Coq-to-Lean compiler-correctness theorem.

The source repository was read at commit

`d56dce013e6264292533efdc5e1de71863c6fb86`.

The computational definitions in `Contender.v`, including the designated fifth contender, are the benchmark throughout this article [2].

2 A precise Lean adaptation of the rules

The original rules require constructive numbers, formal comparison proofs, no axioms, a 15-second definition budget, and a one-minute proof budget. They also forbid defining a new entry by referring to the previous closed contender and ask for a substantial margin [1]. We retain that intent and make the Lean-specific checks explicit.

Requirement	Lean interpretation used here
A natural number	A closed, safe, computable definition of type `Nat`, using ordinary inductive recursors.
Definition budget	At most 15 seconds cumulatively to check all custom computational dependencies from source. The official standard library may be precompiled.
Comparison budget	At most 60 seconds cumulatively for the custom proofs; we additionally include the axiom and dependency audit in the measured proof budget.
No axioms	Empty transitive axiom lists for the entry and the advertised comparison theorems. No `sorry`, declared axioms, classical choice, user-written unsafe definitions, or noncomputable data.
Substantive new entry	No transitive dependence of the submitted number on the previous closed contender. General language definitions and total interpreters remain reusable mathematical infrastructure.
Reproducibility	Pin Lean 4.19.0, disclose the source snapshot, preserve original licensing, and include commands and recorded validation.

The allotted times concern *checking the definition and proof*, not computing the decimal expansion and not the time spent designing the entry. The user supplied no separate deadline for development. We therefore used the repository's explicit checking budgets.

Lean's logical framework includes proof irrelevance as a kernel conversion rule. Additional principles such as propositional extensionality, quotient soundness, and classical choice are visible to the axiom audit. The official manual explains the distinction between pure computation, proof-related extensions, and noncomputable choice [3]. This submission stays within the strict empty-dependency interpretation. Importing a library that contains an unused axiom does not itself make a declaration depend on it.

A few convenience lemmas in Lean's library carry `propext` transitively. The supplied proof files replace the particular maximum and multiplication lemmas that would have introduced that dependency with direct inductive proofs. These replacements are short; there is no need to weaken the contest's no-axioms rule.

3 What Contender 5 actually denotes

3.1 Raw syntax and depth

The source defines simple types by

$$A, B ::= \mathsf{Nat} \mid A \to B$$

and raw terms by

$$t, u ::= \mathsf{var}(i) \mid \lambda^{A,B} t \mid t\, u \mid 0 \mid S \mid \mathsf{rec}_A.$$

The lambda constructor carries both a domain and a codomain annotation. Raw syntax is deliberately not restricted by a typing judgment. Every raw term receives a total, default-valued interpretation.

Type depth is

$$d(\mathsf{Nat}) = 1, \qquad d(A \to B) = 1 + \max(d(A), d(B)).$$

The term-depth conventions are important:

$$\begin{aligned} d(\mathsf{var}(i)) &= i + 2, \\ d(\lambda^{A,B} t) &= 1 + \max(d(A), d(B), d(t)), \\ d(tu) &= 1 + \max(d(t), d(u)), \\ d(0) = d(S) &= 1, \qquad d(\mathsf{rec}_A) = 1 + d(A). \end{aligned}$$

Let $\mathcal{R}_n$ be the exact list produced by the source's `termsUpTo n`. The original file proves that this list contains exactly the raw terms of depth at most n. Our Lean port reproduces its recursion, product nesting, concatenation order, and duplicate behavior literally.

3.2 A total semantic interpreter

The interpretation of a simple type is the corresponding natural-number or function type in the host theory. A value is a dependent pair $\langle A, v \rangle$ with $v \in [\![A]\!]$. The error value is zero at natural type and a constant error function at an arrow type. The interpreter is structural recursion on the raw syntax.

The following details prevent accidental replacement of the benchmark by an easier, different number:

- Variable zero denotes the final environment element. With an environment of length ℓ, variable $i < \ell$ selects position $\ell - 1 - i$. A newly bound value is prepended. Thus indices are levels counting from the outermost binder, not ordinary nearest-binder indices.
- A lambda evaluates its body in the extended environment and casts the result to the annotated codomain. Both annotations affect raw evaluation.
- An application whose head has natural type returns the natural error value. An arrow-valued head casts its argument into the domain before applying the function.
- Equal arrow types use computational, recursively eta-expanded casts. Incompatible types produce the appropriate default value. The definition does not transport values through an opaque equality proof.
- The recursor at A has type $A \to (\mathbb{N} \to A \to A) \to \mathbb{N} \to A$. Its step receives the predecessor index and the previous value.
- After evaluating a closed raw term, a natural-valued result is read as its integer; a function-valued result is read as zero.

These rules cover malformed terms, not just the well-typed fragment usually called Gödel's system T. No normalization theorem for untyped lambda calculus is being assumed. The total host-language definition itself supplies the computation.

3.3 The selected maximum and the numerical envelope

The source traverses $\mathcal{R}_n$, keeping a current best term. It replaces the winner only when the new value is *strictly* greater, so the first winner is retained on ties. Define the numerical envelope

$$E(n) = \mathrm{foldmax}_{t \in \mathcal{R}_n} \mathrm{eval}(t), \qquad \text{with initial maximum } 0.$$

The literal port of the source's closed entry is

$$C_5 = \mathrm{eval}(\mathrm{maxBy}(\mathrm{eval}, 0, 0, \mathcal{R}_{42})).$$

Lemma 3.1 (Selected maximum agrees with the envelope)**.** *For every n, the value selected by the original winner-selection algorithm is $E(n)$. In particular, $C_5 = E(42)$.*

Proof. Use the stronger invariant that the current winning term has value exactly the current maximum m. For an empty list, the selected term has value m, which is the fold's initial accumulator. For a head value v, either $m < v$, in which case the new winner has value $v = \max(m, v)$, or $v \le m$, in which case the existing winner has value $m = \max(m, v)$. Induction on the remaining list completes the proof. The initial winning term is zero and has value zero. The Lean theorem `eval_maxBy` proves the invariant, and `contender_5_eq_envelope` specializes it. □

Enumeration alone grows rapidly. If a_n is the number of types and u_n the number of raw terms, counting the constructors gives

$$a_0 = u_0 = 0, \qquad a_{n+1} = 1 + a_n^2,$$
$$u_{n+1} = n + a_n^2 u_n + u_n^2 + 2 + a_n.$$

The two implementations agree on the following tractable prefix:

Depth	Types	Raw terms	Envelope
0	0	0	0
1	1	2	0
2	2	10	1
3	5	146	2
4	26	24,976	3

For example, $u_{n+1} \ge u_n^2$ and $u_1 = 2$, so $u_{42} \ge 2^{2^{41}}$. This is a lower bound on the size of the list, not a claimed estimate of its largest evaluated number. It explains why expanding the depth-42 list has no role in checking the submission.

4 An independent inflationary base

The new number uses the generic interpreter and envelope, but has no dependency on the closed definition `contender_5`. We define

$$P(0) = E(0), \qquad P(n+1) = P(n) + E(n+1), \qquad b(n) = P(n) + n + 1.$$

Equivalently, $P(n) = \sum_{j=0}^{n} E(j)$. The finite sum provides a convenient monotone envelope without requiring a separate monotonicity proof for the original syntax enumerator.

Proposition 4.1 (Base domination)**.** *For every n, $n + 1 \le b(n)$. For $k \le n$, $E(k) < b(n)$. Consequently $C_5 < b(64)$.*

Proof. Each envelope is a natural number. Hence $P(n) \le P(n+1)$, and $E(k) \le P(k)$. Induction along $k \le n$ yields $E(k) \le P(n) < P(n) + n + 1 = b(n)$. Also $n + 1 \le b(n)$ because $P(n) \ge 0$. Set $k = 42, n = 64$ and use Lemma 3.1. □

The implementation is eleven lines of computational code plus direct order proofs. The numeral 64 is a fixed input and starting higher-type level. The later feedback chooses much larger finite levels from previously computed scores; 64 is not a truncation of those later stages.

5 A constructive amplifier on well-founded trees

The amplification mechanism is independent of the benchmark enumerator. Its input is an explicitly defined function $b : \mathbb{N} \to \mathbb{N}$ satisfying $b(n) \geq n + 1$. Its output is a closed natural number N_b. The construction has two complementary features. Countably branching well-founded trees and operators on higher function spaces provide a substantial supply of recursive structure. A separate feedback mechanism supplies a quantitative growth certificate that does not require assigning ordinal names to those trees. The result is an exact definition together with an independently checkable lower bound.

Throughout this section, iteration means ordinary finite iteration:

$$f^{[0]}(x) = x, \qquad f^{[r+1]}(x) = f^{[r]}(f(x)).$$

This convention exposes the first application of f. It defines the same iterate as the familiar recurrence $f^{[r+1]}(x) = f(f^{[r]}(x))$. The brackets distinguish iteration from exponentiation.

5.1 Trees, evaluation, and substitution

Let $\mathcal{T}$ be the inductively generated type with constructors

$$0_{\mathcal{T}}, \qquad S(a) \quad (a \in \mathcal{T}), \qquad L(f) \quad (f : \mathbb{N} \to \mathcal{T}).$$

The third constructor packages a countable family of immediate subtrees. The word "inductively" is essential: these are well-founded trees, and their eliminator permits structural recursion on a tree, including recursive calls to any member $f(r)$ of a family appearing at a limit node. They are not coinductive trees, and a cyclic equation does not introduce a new inhabitant. We use $0_{\mathcal{T}}$ rather than the numeral 0 when its type might be ambiguous.

For a base $b : \mathbb{N} \to \mathbb{N}$, define

$$F_b(0_{\mathcal{T}}; n) = b(n), \tag{1}$$

$$F_b(S(a); n) = \big(F_b(a; \cdot)\big)^{[n+1]}(n), \tag{2}$$

$$F_b(L(f); n) = F_b(f(n); n). \tag{3}$$

Thus a successor node turns the function supplied by its child into a finite iterator. A limit node selects its nth child. The argument n is allowed to change inside a successor iteration; consequently the evaluator genuinely uses the functions associated with subtrees, rather than merely counting nodes along one fixed path.

For $a, t \in \mathcal{T}$, define the substitution $a \star t$ by replacing every zero leaf of t with a:

$$\begin{aligned} a \star 0_{\mathcal{T}} &= a, \\ a \star S(t) &= S(a \star t), \\ a \star L(f) &= L(r \mapsto a \star f(r)). \end{aligned}$$

The orientation matters: the first argument is inserted into the second. The Lean implementation calls this operation `graft`.

The evaluator returns a function of a natural argument at each tree. Its successor equation can therefore reuse the child function with increasingly large arguments, although the recursive call defining that function is still to a smaller tree. This distinction is useful when reading the termination argument: the natural arguments to child functions need not decrease. Structural

recursion decreases the tree argument, while the separate iteration operation decreases its finite iteration counter. The construction never asks the evaluator to recurse on a tree that it has just generated from itself.

Lemma 5.1 (Inflationarity and the base bound)**.** *If $b(n) \geq n$ for every n, then for every $a \in \mathcal{T}$ and every n,*

$$n \leq b(n) \leq F_b(a; n).$$

If instead $b(n) \geq n+1$ for every n, then $F_b(a; n) \geq n+1$ for every a, n. No monotonicity hypothesis on b is required.

Proof. First observe that $g(x) \geq x$ implies $g^{[r]}(x) \geq x$ for all r, x; this follows by induction on r. Now use structural induction on a. The zero case is equality. In the successor case, put $g(x) = F_b(a; x)$. The induction hypothesis gives $g(x) \geq b(x) \geq x$. Therefore

$$F_b(S(a); n) = g^{[n]}(g(n)) \geq g(n) \geq b(n).$$

At a limit node, use the induction hypothesis for the selected child $f(n)$. This proves the weak statement. The strict statement follows by first weakening $b(n) \geq n + 1$ to $b(n) \geq n$ and then retaining the stronger first inequality in $F_b(a; n) \geq b(n) \geq n + 1$. □

The absence of a monotonicity assumption is more than a simplification of the statement. An inflationary function may take a smaller value at a larger argument; nevertheless its own orbit is nondecreasing, because each successive orbit value is at least its immediate predecessor. The proof uses precisely this orbit property. It does not replace $b(x)$ by $b(y)$ merely because $x \leq y$. This makes the interface applicable to bases assembled from finite searches whose dependence on their search parameter has not separately been proved monotone.

Theorem 5.2 (Substitution law)**.** *For every base b, without an inflationarity assumption, every a, t, and every n,*

$$F_b(a \star t; n) = F_{F_b(a;\cdot)}(t; n).$$

Proof. Proceed by structural induction on t. At a zero leaf both sides are $F_b(a; n)$. For a successor node, the induction hypothesis identifies the two child functions at every natural argument. Pointwise equal functions have pointwise equal finite iterates: prove this auxiliary assertion by induction on the iteration count, replacing the first application and then the remaining iterate. Applying it with count $n + 1$ gives the successor case. At a limit node, both evaluators select the child indexed by n, and the induction hypothesis for that child applies directly. The proof uses pointwise equalities throughout. In particular, the Lean proof does not invoke function extensionality. □

Proposition 5.3. *If $b(n) \geq n$ everywhere, then*

$$F_b(a \star t; n) \geq F_b(a; n) \qquad (a, t \in \mathcal{T}).$$

Proof. Lemma 5.1 says that $F_b(a; \cdot)$ is an inflationary base. Apply the same lemma to that base and to t, and then use Theorem 5.2. □

The substitution law is the main algebraic safeguard in the construction. It permits an elaborate tree to be attached to an existing tree while preserving a proved evaluation bound. The later operators need not satisfy an unproved comparison principle for ordinal notations.

More specifically, t acts as a program transforming the initial unary function supplied at its zero leaves. Substituting a into those leaves replaces that initial function by the already constructed

function $F_b(a;\cdot)$. The theorem identifies these two operations exactly, not merely up to a lower bound. The tree t may depend on a, on an evaluation of a, or on other previously defined data. Once those data have been supplied, the same substitution theorem applies to the resulting pair of trees. That flexibility is what permits the later feedback construction without weakening its proof obligation.

Proposition 5.4 (The zero argument and successor gain)**.** *For every base b and tree a,*

$$F_b(a;0) = b(0).$$

If $b(n) \geq n+1$ everywhere, then

$$F_b(S(a);n) \geq F_b(a;n) + n.$$

Proof. The first assertion follows by tree induction. A successor performs exactly one child application at argument zero, and a limit selects its zeroth child, so both cases reduce to the corresponding induction hypothesis. For the second assertion, an induction on r gives $g^{[r]}(x) \geq x + r$ whenever $g(x) \geq x+1$ for all x. Apply this to $g = F_b(a;\cdot)$, which has the required property by Lemma 5.1, and calculate

$$F_b(S(a);n) = g^{[n]}(g(n)) \geq g(n) + n.$$

The gain vanishes at $n = 0$, exactly as the first assertion requires. □

For a small concrete calibration, take $b(n) = n+1$ and finite successor trees $a_0 = 0_{\mathcal{T}}$, $a_{r+1} = S(a_r)$. Directly from the definition,

$$F_b(a_0;n) = n+1, \qquad F_b(a_1;n) = 2n+1,$$

and

$$F_b(a_2;n) = 2^{n+1}(n+1) - 1.$$

Indeed, the $r$fold iterate of $x \mapsto 2x+1$ at n is $2^r(n+1)-1$, as an induction on r verifies. Substituting $r = n+1$ gives the displayed formula. These elementary calculations concern the successor-only fragment; they do not supply an ordinal interpretation of the general limit constructor. The next tree iterates the last function $n+1$ times. These identities explain the basic acceleration before countable branching or higher function spaces enter the definition. They also show why the evaluation argument is part of the construction, rather than an incidental input.

5.2 Explicit tree operators

We now specify the entire additional tree payload. The following definitions are syntactic operations on $\mathcal{T}$. Their resemblance to familiar ordinal constructions guides the design, but no identification with a Veblen ordinal or an ordinal collapsing function is assumed in any proof.

For a tree transformation $f : \mathcal{T} \to \mathcal{T}$, set

$$O_f(a) = L\big(r \mapsto f^{[r+1]}(a)\big).$$

This *orbit-limit constructor* packages successively longer finite orbits. The name does not assert that it produces a fixed point of an arbitrary f.

Define a tree transformation W by

$$\begin{aligned} W(0_{\mathcal{T}}) &= S(0_{\mathcal{T}}), \\ W(S(a)) &= L\big(r \mapsto (x \mapsto W(a) \star x)^{[r]}(0_{\mathcal{T}})\big), \\ W(L(f)) &= L(r \mapsto W(f(r))). \end{aligned}$$

The implementation names this transformation `omegaPower`. In particular, its successor clause iterates substitution into a zero tree; it is not an appeal to an externally supplied ordinal exponentiation operation.

For a supplied transformation $c : \mathcal{T} \to \mathcal{T}$, define

$$\begin{aligned} E_c(0_{\mathcal{T}}) &= c(0_{\mathcal{T}}), \\ E_c(S(a)) &= c(S(E_c(a))), \\ E_c(L(f)) &= L(r \mapsto E_c(f(r))). \end{aligned}$$

This is an enumeration constructor generated by c. For a countable family $\boldsymbol{f} = (f_0, f_1, \ldots)$ of tree transformations, also define finite prefix applications and repeated sweeps:

$$\begin{aligned} P_{\boldsymbol{f}}(0, a) &= a, & P_{\boldsymbol{f}}(r+1, a) &= f_r(P_{\boldsymbol{f}}(r, a)), \\ Q_{\boldsymbol{f}}(0, a) &= a, & Q_{\boldsymbol{f}}(r+1, a) &= P_{\boldsymbol{f}}(r+1, Q_{\boldsymbol{f}}(r, a)). \end{aligned}$$

The common-limit constructor is

$$C_{\boldsymbol{f}}(a) = L(r \mapsto Q_{\boldsymbol{f}}(r+1, a)).$$

Repeated sweeps deliberately revisit earlier members of the family. This choice is more structured than applying each f_r once and then discarding it. However, our correctness argument only needs the explicit recurrence; it does not require a fixed-point theorem for $C_{\boldsymbol{f}}$.

Define a two-argument transformation V by recursion on its first tree:

$$\begin{aligned} V(0_{\mathcal{T}}, a) &= W(a), \\ V(S(t), a) &= E_{O_{V(t,\cdot)}}(a), \\ V(L(f), a) &= E_{C_{(V(f(r),\cdot))_{r\in\mathbb{N}}}}(a). \end{aligned}$$

Here $O_{V(t,\cdot)}$ is the transformation $x \mapsto O_{V(t,\cdot)}(x)$, and similarly for the common-limit constructor in the last line. The recursive calls $V(t, \cdot)$ and $V(f(r), \cdot)$ involve immediate subtrees of the first argument. Finally, set

$$\gamma = O_{a \mapsto V(a, 0_{\mathcal{T}})}(0_{\mathcal{T}}).$$

The symbol γ denotes this particular tree. It is not a claim that the tree has been proved to represent Γ_0.

These definitions produce one explicit transformation rather than leaving a black-box "large ordinal" in the candidate. Every tree used below is built from the three constructors and the recurrences just displayed. Their role in the size certificate is equally explicit: a resulting tree is placed in the second argument of substitution, where Proposition 5.3 applies regardless of its further behavior.

The dependency order also explains totality of this part of the definition. Substitution is available before W is defined. The orbit, enumeration, prefix, and sweep operations are available before V is defined. In the successor clause of V, the transformation associated with the smaller first tree has already been supplied by structural recursion; enumeration then acts on the second tree using that transformation as a parameter. The limit clause receives the analogous family of transformations for the children of its first tree. There is no recursive call to V on a newly constructed first argument in these clauses.

5.3 A dynamically chosen tower of function spaces

The next layer iterates operators at successively higher function types. Define

$$\mathcal{L}_0 = \mathcal{T}, \qquad \mathcal{L}_{k+1} = \mathcal{L}_k \to \mathcal{L}_k.$$

Thus an element of $\mathcal{L}_{k+2}$ transforms endomorphisms of $\mathcal{L}_k$. Each index is a natural number, and the family itself is defined by natural-number recursion. No operation indexed by an infinite natural number is being introduced.

Successor and countable branching extend pointwise through these spaces. For $x \in \mathcal{L}_k$ and $f : \mathbb{N} \to \mathcal{L}_k$, define

$$\begin{aligned} s_0(x) &= S(x), & s_{k+1}(g)(x) &= s_k(g(x)), \\ \ell_0(f) &= L(f), & \ell_{k+1}(f)(x) &= \ell_k(r \mapsto f(r)(x)). \end{aligned}$$

If $g \in \mathcal{L}_{k+1}$, put

$$\Omega_k(g)(x) = \ell_k\big(r \mapsto g^{[r+1]}(s_k(x))\big).$$

The successor $s_k(x)$ is inserted before the finite orbit. This detail is part of the exact definition and is retained at every function level. The operator Ω_k belongs to $\mathcal{L}_{k+2}$.

Let

$$B_0 = W, \qquad B_{k+1} = \Omega_k.$$

In both cases $B_k \in \mathcal{L}_{k+1}$. A lowering operation converts the result back into a transformation of trees:

$$D_0(f) = f, \qquad D_{k+1}(f) = D_k(f(B_k)).$$

The types match at each step: if $f \in \mathcal{L}_{k+2}$, then $f(B_k) \in \mathcal{L}_{k+1}$. Define the seed family

$$\Sigma_k = D_k(B_k)(\gamma).$$

This family is defined for every natural k. Later, k will itself be an earlier evaluation score plus 64. The dependent type family therefore provides a compact description of a finite function-space tower whose height is determined by the construction, rather than fixed by a long list of manually written arrow types.

The first three seeds make the difference between levels concrete:

$$\begin{aligned} \Sigma_0 &= W(\gamma), \\ \Sigma_1 &= \Omega_0(W)(\gamma) = L(r \mapsto W^{[r+1]}(S(\gamma))), \\ \Sigma_2 &= \Omega_1(\Omega_0)(W)(\gamma). \end{aligned}$$

In the last expression, Ω_0 is itself the function being iterated by the next operator. Expanded one step further, it is

$$\Sigma_2 = L\big(r \mapsto [\Omega_0^{[r+1]}(x \mapsto S(W(x)))](\gamma)\big).$$

Thus the second increase of level is not just another application of W. It iterates the operation that constructs an orbit-limit transformation. Higher levels continue this same typed pattern. The recurrence for D_k ensures that every such operator is eventually applied to appropriate arguments and produces an ordinary member of $\mathcal{T}$.

5.4 Feedback and the final number

Fix the evaluation argument 64, and abbreviate

$$\sigma_b(a) = F_b(a; 64), \qquad t_b(a) = \Sigma_{\sigma_b(a)+64}.$$

The elementary advancement operation is

$$A_b(a) = S(a) \star V\big(a \star t_b(a),\ t_b(a) \star a\big). \tag{4}$$

Both orders of substitution occur in the payload. The outer substitution has first argument $S(a)$; that choice makes the successor gain available before any properties of the payload are investigated.

Let T be the tower of twos with the following exact height convention:

$$T(0) = 1, \qquad T(r+1) = 2^{T(r)}.$$

One complete stage performs a number of advancement operations given by a tower whose height is the current score plus one:

$$G_b(a) = A_b^{[T(\sigma_b(a)+1)]}(a). \tag{5}$$

Starting with $a_0 = \Sigma_{64}$, define

$$a_{r+1} = G_b(a_r), \qquad s_b = \sigma_b(a_0).$$

The proposed output of the amplifier is

$$\boxed{N_b = \sigma_b(a_{s_b}) = F_b(a_{s_b}; 64).} \tag{6}$$

The actual score of the initial tree is therefore also the number of complete feedback stages. At each stage, scores additionally control both a tower height and the level of a function-space construction.

There is no circular definition here. First define tree evaluation and the tree operators. Then define A_b using the already available evaluator on its input tree. Finite iteration defines G_b, natural-number recursion defines a_r, and the already defined initial score supplies the final finite index. During iteration, successive trees can become more complex; termination of that iteration is nevertheless justified by its decreasing natural-number counter. A defining term need not reduce its enormous numeric arguments during type checking.

There are two distinct uses of a score inside a stage. Its total iteration count $T(\sigma_b(a)+1)$ is fixed from the input tree before that stage begins. Within each advancement, the current tree determines a new function-space index for t_b. Recomputing that index does not reset the iteration counter. Likewise, the total number s_b of complete stages is fixed by the initial seed. These distinctions rule out an interpretation as an open-ended loop whose stopping condition continually moves away. Every iteration count is a finite natural number supplied to an ordinary recursor.

5.5 A quantitative certificate independent of calibration

Assume for the remainder of this section that $b(n) \geq n+1$ everywhere. Equation (4), Proposition 5.3, and Proposition 5.4 give, for every evaluation argument n,

$$F_b(A_b(a); n) \geq F_b(S(a); n) \geq F_b(a; n) + n. \tag{7}$$

The payload in A_b uses the score at 64, but the displayed comparison holds at every n. Induction on the number of advancements now yields

$$F_b(A_b^{[r]}(a); n) \geq F_b(a; n) + rn.$$

Writing $m = \sigma_b(a)$ and applying this inequality to one complete stage gives the sharper intermediate estimate

$$\sigma_b(G_b(a)) \geq m + 64T(m+1). \tag{8}$$

The elementary inequalities $2^q \geq q+1$ imply by induction that $T(r) \geq r+1$. They also imply that T is increasing. Consequently (8) has two useful consequences:

$$\sigma_b(G_b(a)) \geq T(m+1), \tag{9}$$
$$\sigma_b(G_b(a)) + 1 \geq 65(m+1). \tag{10}$$

The first follows because $m \geq 0$ and $64 \geq 1$. For the second, use $T(m+1) \geq m+1$ to obtain $m + 64T(m+1) + 1 \geq m + 64(m+1) + 1 = 65(m+1)$.

Define a purely numerical comparison function by

$$\Theta(s, 0) = s, \qquad \Theta(s, r+1) = T(\Theta(s, r) + 1).$$

This repeatedly replaces the previous number by a tower of twos whose height is that number plus one. Its recursion is useful for communicating a guaranteed lower bound: it requires none of the tree operators or higher function spaces.

For example, the first two numerical feedback values are

$$\Theta(s, 1) = T(s+1), \qquad \Theta(s, 2) = T(T(s+1)+1).$$

The second expression uses the entire first tower value as part of a new tower height. The final bound uses s such replacements. It is therefore substantially more explicit than saying only that the candidate contains many iterations: it gives a familiar recursively defined natural function that the actual candidate is proved to exceed.

Theorem 5.5 (Certified feedback growth)**.** *For every $r \in \mathbb{N}$,*

$$\sigma_b(a_r) \geq \Theta(s_b, r),$$
$$\sigma_b(a_r) + 1 \geq 65^r(s_b + 1).$$

In particular,

$$\boxed{N_b \geq \Theta(s_b, s_b)}, \qquad \boxed{N_b + 1 \geq 65^{s_b}(s_b + 1)}.$$

Proof. Both claims are proved by induction on r. At $r = 0$, the first is equality by the definition of s_b, and the second uses $65^0 = 1$. Suppose the first claim holds at r. Monotonicity of T, followed by (9), gives

$$\Theta(s_b, r+1) = T(\Theta(s_b, r) + 1) \leq T(\sigma_b(a_r) + 1) \leq \sigma_b(a_{r+1}).$$

For the second claim, multiply its induction hypothesis by 65 and apply (10). Associativity of multiplication and the successor equation for powers give the required factor 65^{r+1}. Finally specialize both inequalities to $r = s_b$. □

Proposition 5.6 (Transferring a benchmark into the bound)**.** *The function Θ is nondecreasing in each argument. Hence, whenever $c \leq s_b$,*

$$\Theta(c, c) \leq N_b.$$

Also $b(64) \leq s_b$ and $b(64) \leq N_b$.

Proof. Monotonicity in the first argument follows by induction on the second, using monotonicity of T. Since $T(q+1) \geq q+2$, each step of the second argument increases the value, proving its monotonicity as well. Thus $c \leq s_b$ gives $\Theta(c,c) \leq \Theta(s_b,c) \leq \Theta(s_b,s_b) \leq N_b$. The two base comparisons are instances of Lemma 5.1, first for a_0 and then for a_{s_b}. □

The proof distinguishes a guaranteed quantitative margin from a proposed ordinal interpretation. The entire payload appears in the concrete definition of N_b, but the displayed lower bounds remain valid even if its ordinal calibration is left unresolved. They follow from structural recursion, substitution, inflationarity, and explicit finite iteration. This makes the benchmark comparison robust while leaving room for further analysis of how much additional growth the higher operators contribute.

There is a useful modular consequence of the argument. Suppose a later investigation supplies a different explicit total tree transformation $R_b(a)$. Replacing the payload in (4) by $R_b(a)$ while retaining the outer form $S(a)\star R_b(a)$ preserves (7). The induction giving the raw stage estimate and both final bounds then applies without changing its reasoning. The substitution interface therefore isolates a stable mathematical obligation from the choice of additional tree-generating operators. Such a replacement would still require its own well-typed definition and resource checks; the present candidate is the fully specified payload above, rather than an unspecified member of that wider family.

Conversely, the numerical bound alone does not identify the exact value of N_b. The raw stage inequality already discards most information about the generated payload, and the tower comparison discards the additional factor 64 and the previous score. These are deliberate weakenings in a proof of a lower bound. They make the certificate easy to state and transport to a benchmark, while the exact recurrences retain all the additional structure in the proposed number.

6 The closed contender and its certified margin

Specialize the amplifier's base to the function b from Section 4. Let s be its initial seed score and $N_\star = \mathrm{candidate}(b)$. The complete tree construction is fixed by the preceding definitions and reproduced as Lean code in the appendix.

Theorem 6.1 (Strictly beyond Contender 5)**.** *The supplied closed natural number satisfies*

$$C_5 < N_\star.$$

Proof. Proposition 4.1 gives $C_5 < b(64)$. The amplifier's universal base-domination lemma gives $b(64) \leq F_b(a;64)$ for every tree a, in particular for the final feedback tree. Its value is $N_\star$. Transitivity proves the claim. □

The corresponding checked proof is short because all enormous expressions remain symbolic:

```
theorem contender_5_lt_contender_6 :
    Baseline.contender_5 < contender_6 :=
  Nat.lt_of_lt_of_le contender_5_lt_base64
    (Amplifier.base_le_candidate base base_weak)
```

Theorem 6.2 (Incumbent-relative tower guarantee)**.** *Define*

$$T(0) = 1, \qquad T(h+1) = 2^{T(h)}, \qquad \Theta(s,0) = s, \quad \Theta(s,k+1) = T(\Theta(s,k)+1).$$

Then

$$\boxed{\Theta(C_5+1, C_5+1) \le N_\star.}$$

In addition,

$$65^{C_5+1}(C_5+2) \le N_\star + 1.$$

Proof. The initial seed satisfies $s \ge b(64) > C_5$, hence $C_5 + 1 \le s$. The tower function is monotone; induction on the round count shows that $\Theta(s,k)$ is monotone in s. It also increases with the round count because $T(x+1) \ge x+2$. Thus

$$\Theta(C_5+1, C_5+1) \le \Theta(s,s) \le N_\star,$$

where the last inequality is the amplifier's feedback bound. For the second claim, use its independently checked bound $65^s(s+1) \le N_\star + 1$, monotonicity of positive integer powers, and $C_5 + 1 \le s$. □

Both claims are formalized, including the monotonicity needed to replace s by a number expressed in terms of C_5. The power bound is a simple readable consequence. The iterated-tower bound is the stronger headline certificate. These are lower bounds, not asymptotic equivalents or a numerical evaluation of the entry.

There is a practical proof-engineering detail here. Arithmetic lemmas should first be proved for abstract natural-number variables and only then applied to the enormous constants. In one direct closed instantiation, definitional comparison attempted to unfold a large value through an arithmetic alias. The final code uses a generic transfer lemma and, in the exponential bound, spells the right-hand side as `Amplifier.candidate base + 1`. That is definitionally $N_\star + 1$, while avoiding unnecessary closed-value reduction. No mathematical hypothesis or computational definition is changed by this choice.

7 What the tree notation does and does not assert

The constructors named `omegaPower`, `veblenTree`, and `gammaSeed` are explicit tree-building programs. Their names record inspiration from familiar ordinal constructions. The comparison proofs use their actual trees and the grafting identity, not a theorem identifying their ordinal ranks. This distinction is essential for arbitrary raw trees.

Proposition 7.1 (A counterexample to unrestricted ordinal calibration)**.** *Assign an ordinal value to a tree by*

$$\rho(\mathsf{zero}) = 0, \quad \rho(\mathsf{succ}(a)) = \rho(a) + 1, \quad \rho(\mathsf{limit}(f)) = \sup_{n<\omega} \rho(f(n)).$$

The operation `veblenTree` does not, on all raw trees, factor through this ordinal valuation as the standard Veblen function.

Proof. Let $z = \mathsf{zero}$ and $\ell = \mathsf{limit}(n \mapsto z)$. Both have value zero. However, $\mathrm{veblenTree}(z,z) = \mathrm{omegaPower}(z) = \mathsf{succ}(z)$, whose value is one.

For ℓ, the first-argument limit clause uses the common closure of the constant family $a \mapsto \mathrm{omegaPower}(a)$. On input zero, its finite sweep of length r applies this operation $1 + 2 + \cdots + r$ times. These lengths are unbounded. Structural induction shows that omegaPower has valuation $\alpha \mapsto \omega^\alpha$: grafting has valuation ordinal addition, its finite repeated grafts give finite multiples,

and the defining suprema give the successor and limit laws for exponentiation. Hence the common closure at zero has valuation

$$\sup_{j<\omega} \underbrace{\omega^{\omega^{\cdot^{\cdot^{\omega}}}}}_{\text{finitely many exponentiations}} = \varepsilon_0.$$

One output has value one and the other has value ε_0, although their first inputs both have value zero and their second inputs agree. Therefore the construction cannot generally depend only on the two ordinal values in the asserted way. □

This is a mathematical analysis of a possible interpretation, not a Lean ordinal formalization. In particular, the valuation ρ is not strict well-founded tree height: a limit node may have the same value as its children. The example does not affect totality or any numerical theorem in the package. It identifies an assertion that would require additional restrictions and new proofs before being made.

There is a second useful check: the formal theorem `fast_at_zero` says $F_b(a;0) = b(0)$ for *every* tree a. At a successor the iteration count is one, and at a limit only branch zero is selected. Thus even extremely complicated tree syntax supplies no strict gain at input zero. Numerical dominance must follow from the evaluator's rules and hypotheses; ordinal language alone cannot establish it. Our entry evaluates at 64, where each explicit successor provides the proved positive gain.

8 Validation and reproducibility

8.1 What was checked

The archive contains a validator that stages the required Lean sources in a fresh directory, discovers their local imports, and rebuilds every custom dependency. It removes inherited project search paths, so pre-existing project `.olean` files cannot silently supply the entry or proofs. Only the pinned official toolchain's standard library is reused.

The definition phase contains `BaselineDefs`, `BaseDefs`, `AmplifierDefs`, and `CandidateDefs`. The proof phase contains `Baseline`, `Base`, `Amplifier`, and `Candidate`. An additional audit computes transitive axiom dependencies for 32 selected public declarations, including all final comparison theorems. Every list is empty. A declaration-body dependency traversal also verifies that `contender_6` does not depend on the closed fifth contender or either of the source's selected-winner wrappers. It does use the general raw-language interpreter and numerical envelope, as intended.

Phase	Measured seconds	Budget (seconds)
All custom definitions	1.408	15
All custom proofs	1.363	60
Proofs plus axiom/dependency audits	2.520	60
Additional stored-body kernel check	2.458	separate

The required eight source modules contain 697 lines and 25,777 UTF-8 bytes. Compiler processes used one thread on an AMD EPYC 9V74 host running Ubuntu 24.04.3. The peak resident memory during definition/proof compilation was 428.3 MiB; including the extra checks, the peak was 927.9 MiB. The additional check resubmitted 264 safe stored custom definition and theorem bodies directly to the C++ kernel under fresh names. It retained the imported standard-library declarations and inductive metadata; it is a body check, not a separate proof of the Lean implementation.

Compiler-generated unsafe execution helpers were excluded, while the corresponding safe logical definitions were checked. The recorded scripts explain the precise scope.

These are observed wall times on the recorded environment, not hardware- independent performance guarantees. The validator supports the same numerical budgets on the reader's machine and fails rather than silently relaxing them. No enormous natural number is evaluated during either budgeted phase.

The toolchain is Lean 4.19.0, commit

`6caaee842e9495688c1567e78c0e68dbb96942aa`.

The official compiler and kernel libraries were unchanged. This container required a small disclosed application-path launcher because its ordinary executable-path query was unavailable. The launcher supplies its own resolved program path; it does not modify the elaborator or kernel. Its complete C source, build instructions, release archive hash, and shared-library hash are included in `scripts/REPRODUCTION.md` and the adjacent source files. An ordinary official Lean installation does not need this launcher.

8.2 Independent semantic checks

The Python model in `scripts/reference_model.py` independently implements the original raw syntax, cast pairing, reversed lookup, and recursor. Its depth-zero-through-four rows agree with Lean's executable smoke tests. Both implementations also check an example of recursion returning a function: start with successor, double function composition three times, and apply the resulting function to zero; the answer is eight. The Lean version is a closed reduction proof. It uses a locally increased reduction-depth limit only in this optional test, not in the entry or comparison modules.

The independent model is a translation check, not a second proof assistant. Likewise, finite tests cannot prove that two implementations agree at all inputs. The universal comparison theorem is supplied by Lean, and the cross-language correspondence is supported by the detailed source audit and literal preservation of the relevant computational definitions.

8.3 Commands

With the pinned Lean toolchain available, run from the archive root:

```
python3 scripts/validate.py --lean lean
python3 scripts/reference_model.py
lake build
```

The first command is the budgeted, clean-source validation. The third is a convenient ordinary project build; it is not the source of the timing claim. The optional executable smoke modules are deliberately outside the submitted entry's import graph.

To rebuild this article, run

```
cd article
latexmk -xelatex -interaction=nonstopmode -halt-on-error \
  Lean_Big_Number.tex
```

The source uses standard TeX packages, Latin Modern fonts, and DejaVu Sans Mono for Unicode Lean listings. All input fragments are supplied.

9 Constructivity, the diagonal barrier, and optimality

9.1 Why the definition is a natural number

Every component is a total definition in the constructive fragment used by the submission. The language enumerators recurse on natural depth. Interpretation recurses on syntax and uses ordinary natural-number recursors at interpreted types. The tree evaluator recurses on the inductive tree; its extra iterations have finite natural counts. Higher function spaces are formed by finite recursion on a natural level. Feedback uses a value from a previously constructed tree to choose a new finite count or level. There is no equation that defines a tree or natural number by presupposing its own final value.

A limit constructor takes a function $\mathbb{N} \to \mathsf{Tree}$. This does not require writing infinitely many subtrees into a file. A finite, total program provides the children. Structural recursion receives each requested child's recursive result, as licensed by the inductive recursor. A cyclic definition would not pass the same termination check.

The finite maximum defining $E(n)$ ranges over a language with an explicitly given total evaluator. It does not search arbitrary programs and wait for an unknown set of halting computations. Accordingly, the entry does not depend on a Busy Beaver bound, a halting oracle, classical selection of an unspecified maximum, or an unproved consistency assumption.

9.2 Why one cannot simply maximize over all Lean definitions

A tempting shortcut is to replace the object-language envelope by a purported total evaluator for every total Lean function, then diagonalize. That step requires a justification which the present construction deliberately does not assume. The elementary obstruction is worth stating precisely.

Proposition 9.1 (Diagonal obstruction)**.** *Suppose an enumerated class of total unary functions has a total evaluator $U(e, n)$, represents every function in that class by some code, and is closed under the function $d(n) = U(n, n) + 1$ defined in the same class. These assumptions are inconsistent.*

Proof. If e represents d, then $d(e) = U(e, e) + 1 = d(e) + 1$, impossible in $\mathbb{N}$. □

This proposition does not rule out total interpretation of a weaker object language inside a stronger host language. That is exactly the relationship used here: the raw system-T-style language is interpreted in Lean, and the new tree feedback is an additional host-language construction. The argument also does not forbid fixed, finite maxima with independently justified total interpreters. It identifies why a supposed universal same-language evaluator cannot be inserted without resolving its scope and totality obligations.

9.3 The meaning of "largest"

This article proposes a concrete entry and proves its performance against the specified incumbent. It does not establish that every admissible Lean program names a smaller number. If a fixed machine, time bound, and finite source-size cap produce a finite set of accepted source strings, then that finite contest has a maximum score abstractly. The work here neither characterizes that set nor proves optimality over it.

The elementary successor operation explains why there is no largest natural number, and can improve a particular submission when sufficient resource slack remains. It is not, by itself, a proof that a resource-bounded contest has no maximum. Nor would it justify presenting a circular "one greater than every permissible entry" description as a constructive Lean definition. The present

submission has an explicit source, a total computation, and a checked strict margin; those are the claimed achievements.

10 Further research directions

1. **A calibrated domain of ordinal trees.** Identify a decidable or constructively checkable class of well-behaved trees on which the closure constructors respect an ordinal valuation. The degenerate-limit counterexample shows that restrictions must be stated, not inferred from notation. A useful result would combine closure of the class under the actual feedback operators with a proved numerical comparison principle.
2. **Direct majorization of the raw interpreter.** Prove bounds for $E(n)$ in a separately specified fast-growing hierarchy, including the effects of type annotations, recursor result types, malformed terms, and default casts. Such a theorem could replace the exhaustive envelope in the base by a more transparent closed growth function while retaining a checked comparison with the exact fifth contender.
3. **A richer certified object language.** Extend the finite syntax with one explicit inductive tree type and its recursor, give a total interpretation, and prove an embedding of the current raw language with a controlled depth overhead. Iterating carefully stratified language extensions could provide a different route to stronger contenders without positing a universal Lean self-interpreter.
4. **A verified translation bridge.** Formalize enough of the Coq computational subset to certify the correspondence of the two implementations. An intermediate first-order syntax for types, raw terms, casts, and evaluator equations would make the remaining cross-language trust boundary explicit.
5. **Sharper structural lower bounds.** The graft proof intentionally ignores most of the Veblen-inspired payload. Find syntax-sensitive order relations preserved by evaluation at controlled inputs, and use them to quantify the higher-type seeds. The equality at input zero warns against unqualified pointwise monotonicity claims.
6. **Checking efficiency with strict dependency audits.** Explore proof organization, abstract interfaces, and kernel conversion behavior while preserving empty axiom lists. The closed-value alias issue illustrates why symbolic theorem design can matter more than the physical length of a proof. Record total custom dependency cost, not just the final declaration's cost.

A Archive map and provenance

Path	Purpose
`BigNumber/BaselineDefs.lean`	Literal computational port and numerical envelope.
`BigNumber/Baseline.lean`	Selected-max invariant and axiom-free maximum proofs.
`BigNumber/BaseDefs.lean`	Inclusive envelope sum and independent base.
`BigNumber/Base.lean`	Base domination, including the exact incumbent.
`BigNumber/AmplifierDefs.lean`	Trees, interpreter, closure constructors, higher types, feedback.

Path	Purpose
`BigNumber/Amplifier.lean`	Grafting, gains, monotonicity, and quantitative lower bounds.
`BigNumber/CandidateDefs.lean`	The submitted closed natural number.
`BigNumber/Candidate.lean`	Final strict and incumbent-relative certificates.
`BigNumber/BaselineSmoke.lean`	Optional executable small-depth tests.
`BigNumber/SemanticChecks.lean`	Closed higher-type recursor reduction test.
`scripts/validate.py`	Clean build, cumulative budgets, axiom and dependency audit.
`scripts/reference_model.py`	Independent small-input semantic model.
`validation/`	Recorded outcomes, logs, and independent mathematical review.
`upstream/`	Pinned Coq source, original rules, and MIT license.
`article/`	Complete article source, fragments, and compiled PDF.

The upstream `Contender.v` Git blob is `f92db72deb0f3a4d4e3b575b4949eda73b1417a0`. The exact source files and their SHA-256 hashes are recorded in the archive manifest. The original source is redistributed under its MIT license, retained in `upstream/LICENSE`. New material is separately identified in the root README and license.

B Complete computational amplifier

This listing is the exact computational module used in validation. It has no custom imports; the standard Lean prelude is implicit. The baseline and base modules are supplied as separate source files.

```
namespace BigNumber.Amplifier

/-- Finite iteration, with the initial application exposed definitionally. -/
def iterate {α : Type u} (f : α → α) : Nat → α → α
  | 0, x => x
  | n + 1, x => iterate f n (f x)

/-- A countably branching, inductively well-founded tree. -/
inductive Tree : Type where
  | zero : Tree
  | succ : Tree → Tree
  | limit : (Nat → Tree) → Tree

/-- The complete evaluator. All recursive calls are to immediate subtrees. -/
def fast (b : Nat → Nat) : Tree → Nat → Nat
  | .zero, n => b n
  | .succ a, n => iterate (fast b a) (n + 1) n
  | .limit f, n => fast b (f n) n

/-- Substitute `a` for every zero leaf of the second argument. -/
def graft (a : Tree) : Tree → Tree
  | .zero => a
  | .succ t => .succ (graft a t)
  | .limit f => .limit (fun n => graft a (f n))

/-- A syntactic orbit supremum; no quotient by ordinal equality is used. -/
def orbitLimit (f : Tree → Tree) (a : Tree) : Tree :=
  .limit (fun n => iterate f (n + 1) a)

def omegaPower : Tree → Tree
```

```
  | .zero => .succ .zero
  | .succ a => .limit (fun n => iterate (graft (omegaPower a)) n .zero)
  | .limit f => .limit (fun n => omegaPower (f n))

/-- Enumeration generated by a supplied closure constructor. -/
def enumClosure (c : Tree → Tree) : Tree → Tree
  | .zero => c .zero
  | .succ a => c (.succ (enumClosure c a))
  | .limit f => .limit (fun n => enumClosure c (f n))

def prefixApply (fs : Nat → Tree → Tree) : Nat → Tree → Tree
  | 0, a => a
  | n + 1, a => fs n (prefixApply fs n a)

def sweep (fs : Nat → Tree → Tree) : Nat → Tree → Tree
  | 0, a => a
  | n + 1, a => prefixApply fs (n + 1) (sweep fs n a)

def commonLimit (fs : Nat → Tree → Tree) (a : Tree) : Tree :=
  .limit (fun n => sweep fs (n + 1) a)

/-- Explicit Veblen-inspired syntax, without an ordinal calibration assertion. -/
def veblenTree : Tree → Tree → Tree
  | .zero, a => omegaPower a
  | .succ t, a => enumClosure (orbitLimit (veblenTree t)) a
  | .limit f, a => enumClosure (commonLimit (fun n => veblenTree (f n))) a

def gammaSeed : Tree := orbitLimit (fun a => veblenTree a .zero) .zero

/-- A finite but dynamically selectable tower of higher function spaces. -/
def Level : Nat → Type
  | 0 => Tree
  | k + 1 => Level k → Level k

def pointSucc : (k : Nat) → Level k → Level k
  | 0, a => .succ a
  | k + 1, f => fun x => pointSucc k (f x)

def pointLimit : (k : Nat) → (Nat → Level k) → Level k
  | 0, f => .limit f
  | k + 1, fs => fun x => pointLimit k (fun n => fs n x)

def omegaIteration (k : Nat) (f : Level (k + 1)) : Level (k + 1) :=
  fun x => pointLimit k (fun n => iterate f (n + 1) (pointSucc k x))

def baseOperator : (k : Nat) → Level (k + 1)
  | 0 => omegaPower
  | k + 1 => omegaIteration k

def lower : (k : Nat) → Level (k + 1) → Tree → Tree
  | 0, f => f
  | k + 1, f => lower k (f (baseOperator k))

def higherSeed (k : Nat) : Tree := lower k (baseOperator k) gammaSeed

def advance (b : Nat → Nat) (a : Tree) : Tree :=
  let t := higherSeed (fast b a 64 + 64)
  graft (.succ a) (veblenTree (graft a t) (graft t a))

/-- An explicit tower of twos, with height supplied by an earlier score. -/
def tower2 : Nat → Nat
  | 0 => 1
  | k + 1 => 2 ^ tower2 k

def stage (b : Nat → Nat) (a : Tree) : Tree :=
  iterate (advance b) (tower2 (fast b a 64 + 1)) a

def ladder (b : Nat → Nat) : Nat → Tree
```

```
  | 0 => higherSeed 64
  | k + 1 => stage b (ladder b k)

def seedScore (b : Nat → Nat) : Nat := fast b (higherSeed 64) 64

/-- The proposed score, parameterized only by an explicitly supplied base. -/
def candidate (b : Nat → Nat) : Nat := fast b (ladder b (seedScore b)) 64

/-- A transparent numeric lower bound for the full tree construction. -/
def towerFeedback (seed : Nat) : Nat → Nat
  | 0 => seed
  | k + 1 => tower2 (towerFeedback seed k + 1)

end BigNumber.Amplifier
```

C The base and final comparison proofs

The following are the exact base and final proof modules. All supporting proofs, including the full graft and feedback theorems, are supplied in the archive and explained in the main text.

C.1 Base definitions

```
import BigNumber.BaselineDefs

namespace BigNumber

/-- Inclusive running sum of the total-language envelopes. -/
def prefixEnvelope : Nat → Nat
  | 0 => Baseline.envelope 0
  | n + 1 => prefixEnvelope n + Baseline.envelope (n + 1)

/-- A constructive, inflationary base defined independently of any contender. -/
def base (n : Nat) : Nat := Nat.succ (prefixEnvelope n + n)

end BigNumber
```

C.2 Base proofs

```
import BigNumber.BaseDefs
import BigNumber.Baseline

namespace BigNumber

theorem envelope_le_prefix (n : Nat) :
    Baseline.envelope n ≤ prefixEnvelope n := by
  cases n with
  | zero => exact Nat.le_refl _
  | succ n => exact Nat.le_add_left _ _

theorem prefix_mono {n m : Nat} (h : n ≤ m) :
    prefixEnvelope n ≤ prefixEnvelope m := by
  induction h with
  | refl => exact Nat.le_refl _
  | @step m h ih => exact Nat.le_trans ih (Nat.le_add_right _ _)

theorem base_inflationary (n : Nat) : n < base n :=
  Nat.lt_of_le_of_lt (Nat.le_add_left n (prefixEnvelope n))
    (Nat.lt_succ_self _)

theorem prefix_lt_base (n : Nat) : prefixEnvelope n < base n :=
  Nat.lt_of_le_of_lt (Nat.le_add_right (prefixEnvelope n) n)
    (Nat.lt_succ_self _)

```

```lean
theorem envelope_lt_base {k n : Nat} (h : k ≤ n) :
    Baseline.envelope k < base n :=
  Nat.lt_of_le_of_lt
    (Nat.le_trans (envelope_le_prefix k) (prefix_mono h))
    (prefix_lt_base n)

theorem contender_5_lt_base64 : Baseline.contender_5 < base 64 := by
  rw [Baseline.contender_5_eq_envelope]
  exact envelope_lt_base (by decide)

end BigNumber
```

C.3 Closed definition

```lean
import BigNumber.BaseDefs
import BigNumber.AmplifierDefs

namespace BigNumber

/-- The submitted closed natural number. It has no dependency on contender_5. -/
def contender_6 : Nat := Amplifier.candidate base

end BigNumber
```

C.4 Final certificates

```lean
import BigNumber.CandidateDefs
import BigNumber.Base
import BigNumber.Amplifier

namespace BigNumber

theorem base_weak (n : Nat) : n ≤ base n := Nat.le_of_lt (base_inflationary n)

theorem base_strict (n : Nat) : n + 1 ≤ base n := base_inflationary n

theorem contender_5_lt_seedScore :
    Baseline.contender_5 < Amplifier.seedScore base :=
  Nat.lt_of_lt_of_le contender_5_lt_base64
    (Amplifier.base_le_fast base base_weak (Amplifier.higherSeed 64) 64)

/-- The strict comparison required by the adapted contest. -/
theorem contender_5_lt_contender_6 : Baseline.contender_5 < contender_6 :=
  Nat.lt_of_lt_of_le contender_5_lt_base64
    (Amplifier.base_le_candidate base base_weak)

/-- Keep arithmetic symbolic before substituting the enormous constants. -/
theorem exponential_transfer (c s v : Nat) (hc : c < s)
    (hv : 65 ^ s * (s + 1) ≤ v + 1) :
    65 ^ (c + 1) * (c + 2) ≤ v + 1 := by
  have hs : c + 1 ≤ s := hc
  exact Nat.le_trans
    (Nat.mul_le_mul (Nat.pow_le_pow_right (by decide) hs)
      (Nat.add_le_add_right hs 1))
    hv

/-- A quantitative lower bound, expressed directly in terms of the incumbent. -/
theorem contender_6_exponential_bound :
    65 ^ (Baseline.contender_5 + 1) * (Baseline.contender_5 + 2) ≤
      Amplifier.candidate base + 1 :=
  exponential_transfer Baseline.contender_5 (Amplifier.seedScore base)
    (Amplifier.candidate base) contender_5_lt_seedScore
    (Amplifier.candidate_bound base base_strict)

theorem tower_transfer (b : Nat → Nat) (hb : ∀ n, n + 1 ≤ b n)
    (c : Nat) (hc : c < Amplifier.seedScore b) :
```

```
    Amplifier.towerFeedback (c + 1) (c + 1) ≤ Amplifier.candidate b :=
  Amplifier.candidate_tower_from b hb (c + 1) hc

/-- A tower is built, then its value becomes the next tower height.
The process repeats more times than the incumbent's value.
-/
theorem contender_6_tower_bound :
    Amplifier.towerFeedback (Baseline.contender_5 + 1)
      (Baseline.contender_5 + 1) ≤ contender_6 :=
  tower_transfer base base_strict Baseline.contender_5 contender_5_lt_seedScore

theorem contender_6_seed_tower_bound :
    Amplifier.towerFeedback (Amplifier.seedScore base)
      (Amplifier.seedScore base) ≤ contender_6 :=
  Amplifier.candidate_tower base base_strict

end BigNumber
```

References

[1] Cody Roux and contributors. *name-the-biggest-number: contest rules*. Pinned repository snapshot, accessed 12 October 2026. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/README.md.

[2] Cody Roux, Samuel Gruetter, and contributors. *Contender.v*, especially the raw-language evaluator and Contender 5. Pinned repository snapshot. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/Contender.v.

[3] Lean community. *Theorem Proving in Lean 4*, chapter "Axioms and Computation." Accessed 12 October 2026. https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/.

[4] Lean development team. *Lean 4.19.0*, pinned release and implementation. https://github.com/leanprover/lean4/releases/tag/v4.19.0. The exact implementation commit and library hashes used in this work appear in the reproduction record.